FirstOffer

DATA ANALYST INTERVIEW KIT · PYTHON

20 Python & Pandas Questions for Data Analysts

The data-wrangling questions interviewers use to check whether you've actually worked with data — cleaning, grouping, merging, dates and quick analysis — answered the way you'd say it in the room.

Pandas basics | Cleaning | GroupBy & Merge | Dates | Analysis

20 questions

20 tested examples

5 topics

firstoffer.online

Fresher jobs · Direct apply links · No middlemen

Written from the interviewer's side · Easy → Medium

START HERE

How to use this kit

1
Read the question and answer it out loud before you look at the model answer.

2
Read “What the interviewer is checking” — that's what you're actually scored on.

3
Run the code yourself, change one thing, and predict the new output before running it again.

SPOT THE CLUE

If the question says… reach for…

“First look at a new dataset”	**shape · head() · info() · describe() · isna().sum()**
“Pick rows / columns”	**loc (labels) · iloc (positions) · boolean masks**
“Totals per group”	**groupby().agg() · pivot_table()**
“Combine two tables”	**merge (SQL-style join) · concat (stack)**
“Clean messy data”	**fillna / dropna · drop_duplicates · str methods · astype**
“Anything with dates”	**pd.to_datetime · .dt accessor · resample**
“Trends”	**rolling() · pct_change() · cumsum()**
“Speed”	**vectorised ops > apply > Python loops**

IN THE INTERVIEW ROOM

1 · Ask what the data looks like **2 · Check nulls & duplicates** **3 · Say the steps in English**

4 · Write the code **5 · Sanity-check the output** **6 · Explain the business meaning**

SAMPLE DATA

The data used in this kit

Every snippet below starts from this small `sales` table (8 orders, one missing amount).

PYTHON

```python
import pandas as pd
import numpy as np

sales = pd.DataFrame({
    "order_id": [1, 2, 3, 4, 5, 6, 7, 8],
    "customer": ["Ravi", "Neha", "Ravi", "Imran", "Ravi", "Pooja", "Neha", "Imran"],
    "city":     ["Bengaluru", "Pune", "Bengaluru", "Delhi",
                 "Bengaluru", "Bengaluru", "Pune", "Delhi"],
    "category": ["Electronics", "Books", "Books", "Grocery",
                 "Electronics", "Electronics", "Books", "Grocery"],
    "order_date": ["2026-01-05", "2026-01-18", "2026-02-02", "2026-02-11",
                   "2026-03-09", "2026-03-15", "2026-03-28", "2026-03-30"],
    "amount":   [1200, 800, 1500, 400, 700, 2200, 950, np.nan],
})
```

CONTENTS

20 questions, easy to medium

Python basics for analysts

01	List vs tuple vs set vs dictionary — when do you use each?	Easy
02	Use a list comprehension to clean and filter a list.	Easy
03	Count how often each word appears in a sentence.	Easy
04	What is a lambda function? Where would an analyst use one?	Easy

First look & selecting data

05	You get a new dataset. What are the first things you check?	Easy
06	What's the difference between loc and iloc?	Easy
07	Filter rows with more than one condition.	Easy

Cleaning data

08	How do you handle missing values?	Easy
09	How do you find and remove duplicate rows?	Easy
10	Clean a messy text column (spaces, cases, typos).	Easy
11	Create a column based on conditions.	Easy

Grouping, merging & reshaping

12	Find total, average and number of orders per city.	Easy
13	How do merge, join and concat differ?	Easy
14	Make a city × category revenue table.	Medium
15	Convert a wide table (one column per month) to long format.	Medium
16	Get the top order in each category.	Medium

Dates, trends & quick analysis

17	Find monthly revenue from an order-date column.	Medium
18	Show the month-over-month % change and a running total.	Medium
19	What share of orders does each category have?	Easy
20	How do you detect outliers in a numeric column?	Medium

Python basics for analysts

01 List vs tuple vs set vs dictionary — when do you use each?

Python basics · Easy

WHAT THE INTERVIEWER IS CHECKING

Whether you pick the right structure instead of using lists for everything.

MODEL ANSWER

- List: ordered and changeable — a sequence of values you'll add to or sort.
- Tuple: ordered but fixed — a record that shouldn't change, like (lat, long). Can be a dict key.
- Set: unique values only, very fast 'is x in here?' checks — great for de-duplicating.
- Dictionary: key → value lookups, like customer → total spend.

PYTHON

```
cities = ["Pune", "Delhi", "Pune", "Mumbai"]
print(set(cities))                      # unique cities
spend = {"Ravi": 3400, "Neha": 1750}
spend["Pooja"] = 2200                   # add a key
print(spend.get("Imran", 0))            # safe lookup with default
point = (12.97, 77.59)                  # tuple: fixed record
print(len(cities), len(spend), point[0])
```

OUTPUT ✓ REAL RUN

```
{'Delhi', 'Pune', 'Mumbai'}
0
4 3 12.97
```

Common mistake Checking membership in a big list inside a loop — use a set, it's far faster.

Interviewer may ask next Why can a tuple be a dictionary key but a list can't? (Keys must be immutable/hashable.)

02 Use a list comprehension to clean and filter a list.

Python basics | Easy

WHAT THE INTERVIEWER IS CHECKING

Comfort with compact, readable Python.

MODEL ANSWER

- Pattern: [expression for item in items if condition].
- Here: keep amounts above 500 and add 18% GST, rounded.
- A dict comprehension works the same way: {k: v for ...}.

PYTHON

```
amounts = [1200, 800, 400, 2200, 350]
with_gst = [round(a * 1.18) for a in amounts if a > 500]
print(with_gst)

names = ["  ravi ", "NEHA", "Imran  "]
print({n.strip().title(): len(n.strip()) for n in names})
```

OUTPUT ✓ REAL RUN

```
[1416, 944, 2596]
{'Ravi': 4, 'Neha': 4, 'Imran': 5}
```

Common mistake Writing a 3-level nested comprehension nobody can read — use a normal loop then.

Interviewer may ask next What's the difference between a list comprehension and a generator expression? (Generator is lazy, uses ( ) and saves memory.)

03 Count how often each word appears in a sentence.

Python basics · Easy

WHAT THE INTERVIEWER IS CHECKING

Dictionaries + string handling — a very common warm-up.

MODEL ANSWER

- Lower-case and split the text, then count with a dictionary.
- collections.Counter does the same in one line and has most_common().

PYTHON

```
from collections import Counter

text = "Data beats opinions and data tells stories"
counts = {}
for word in text.lower().split():
    counts[word] = counts.get(word, 0) + 1
print(counts["data"])

print(Counter(text.lower().split()).most_common(2))
```

OUTPUT ✓ REAL RUN

```
2
[('data', 2), ('beats', 1)]
```

Common mistake Forgetting to lower-case, so 'Data' and 'data' are counted separately.

Interviewer may ask next How would you also strip punctuation? (str.translate or a regex like re.findall(r'\w+', text).)

04 What is a lambda function? Where would an analyst use one?

Python basics · Easy

WHAT THE INTERVIEWER IS CHECKING

Knowing lambdas are small throwaway functions, often used with sort or apply.

MODEL ANSWER

- A lambda is a one-line anonymous function: lambda x: x * 2.
- Common uses: a sort key, or a quick transform inside pandas apply/map.
- If the logic needs more than one line, write a normal def with a name.

PYTHON

```
orders = [("Ravi", 3400), ("Imran", 800), ("Pooja", 2200)]
print(sorted(orders, key=lambda o: o[1], reverse=True))

def band(amount):
    return "High" if amount >= 1000 else "Low"

print(sales["amount"].dropna().map(band).tolist())
```

OUTPUT ✓ REAL RUN

```
[('Ravi', 3400), ('Pooja', 2200), ('Imran', 800)]
['High', 'Low', 'High', 'Low', 'Low', 'High', 'Low']
```

Common mistake Cramming complex if/else logic into a lambda that's impossible to debug.

Interviewer may ask next What do map(), filter() and zip() do?

First look & selecting data

05 You get a new dataset. What are the first things you check?

EDA Easy

WHAT THE INTERVIEWER IS CHECKING

A calm, repeatable first-look routine — this shows real experience.

MODEL ANSWER

- Size and columns: shape, head(), dtypes / info().
- Missing values per column: isna().sum().
- Summary stats for numbers (describe) and unique counts for categories (nunique).
- Then sanity checks: duplicates, impossible values (negative amounts), date ranges.

PYTHON

```
print(sales.shape)
print(sales.isna().sum()[lambda s: s > 0])   # only columns with nulls
print(sales["amount"].describe().round(1)[["count", "mean", "min", "max"]])
print(sales["city"].nunique(), "cities")
```

OUTPUT ✓ REAL RUN

```
(8, 6)
amount    1
dtype: int64
count       7.0
mean     1107.1
min       400.0
max      2200.0
Name: amount, dtype: float64
3 cities
```

Common mistake Jumping straight into charts before checking nulls and data types.

Interviewer may ask next The amount column loaded as text (object). How do you fix it? (pd.to_numeric(col, errors='coerce').)

06 What's the difference between loc and iloc?

Selecting data | Easy

WHAT THE INTERVIEWER IS CHECKING

One of the most-asked pandas questions.

MODEL ANSWER

- loc selects by label — row index labels and column names. Slices include the end label.
- iloc selects by integer position, like Python lists. Slices exclude the end.
- loc also takes boolean masks, which is how you filter and update safely.

PYTHON

```
print(sales.loc[0:2, ["customer", "amount"]])   # labels 0,1,2 → 3 rows
print(sales.iloc[0:2, [1, 5]])                   # positions 0,1 → 2 rows
```

OUTPUT ✓ REAL RUN

```
  customer  amount
0     Ravi  1200.0
1     Neha   800.0
2     Ravi  1500.0
  customer  amount
0     Ravi  1200.0
1     Neha   800.0
```

Common mistake Expecting loc[0:2] to return 2 rows — loc includes the end label.

Interviewer may ask next How do you update amounts for Pune only? (sales.loc[sales['city'] == 'Pune', 'amount'] = ...)

07 Filter rows with more than one condition.

Selecting data | Easy

WHAT THE INTERVIEWER IS CHECKING

Using & / | with brackets — a classic stumbling block.

MODEL ANSWER

- Use & (and), | (or), ~ (not) — not the Python words and/or.
- Wrap every condition in brackets because & binds tighter than ==.
- isin() is cleaner than many | conditions; query() is a readable alternative.

PYTHON

```
big_blr = sales[(sales["city"] == "Bengaluru") & (sales["amount"] > 1000)]
print(big_blr[["order_id", "amount"]])

print(sales[sales["category"].isin(["Books", "Grocery"])]["order_id"].tolist())
```

OUTPUT ✓ REAL RUN

```
   order_id  amount
0         1  1200.0
2         3  1500.0
5         6  2200.0
[2, 3, 4, 7, 8]
```

Common mistake sales[sales.city == 'Pune' and sales.amount > 500] → 'truth value of a Series is ambiguous' error.

Interviewer may ask next Write the first filter with .query().

Cleaning data

08 How do you handle missing values?

Cleaning Easy

WHAT THE INTERVIEWER IS CHECKING

That you think about WHY data is missing before filling it.

MODEL ANSWER

- First measure it: isna().sum() and the % missing per column.
- Options: drop rows (few missing, random), fill with median/mode/0 (when it makes business sense), or flag it.
- Median is safer than mean when the column has outliers.
- Never fill silently — mention the choice in your analysis.

PYTHON

```
print(sales["amount"].isna().mean() * 100, "% missing")
filled = sales["amount"].fillna(sales["amount"].median())
print(filled.iloc[-1])
print(len(sales.dropna(subset=["amount"])), "rows after dropping")
```

OUTPUT ✓ REAL RUN

```
12.5 % missing
950.0
7 rows after dropping
```

Common mistake Filling every gap with 0 — a missing order amount is not a ₹0 order.

Interviewer may ask next When would you fill missing values per group (e.g. median per city)? (groupby + transform('median').)

09 How do you find and remove duplicate rows?

Cleaning Easy

WHAT THE INTERVIEWER IS CHECKING

Knowing subset= and keep= — duplicates are rarely full-row copies.

MODEL ANSWER

- duplicated() marks repeats; drop_duplicates() removes them.
- Use subset= for the columns that define 'the same record', and keep='first' or 'last'.

PYTHON

```
dupes = pd.concat([sales, sales.iloc[[2]]], ignore_index=True)  # add a copy
print(dupes.duplicated().sum(), "exact duplicate")
clean = dupes.drop_duplicates(subset=["order_id"], keep="first")
print(len(dupes), "->", len(clean))
```

OUTPUT ✔ REAL RUN

```
1 exact duplicate
9 -> 8
```

Common mistake Calling drop_duplicates() and forgetting to assign the result back — it returns a new DataFrame.

Interviewer may ask next Keep the latest record per customer instead (sort by date, then drop_duplicates('customer', keep='last')).

10 Clean a messy text column (spaces, cases, typos).

Cleaning Easy

WHAT THE INTERVIEWER IS CHECKING

Using the vectorised .str methods instead of loops.

MODEL ANSWER

- .str.strip() removes extra spaces, .str.lower()/.str.title() fixes case.
- .str.replace() and a mapping dict fix known variants (e.g. 'Bangalore' → 'Bengaluru').
- .str.contains() filters by pattern.

PYTHON

```
raw = pd.Series(["  bengaluru", "Bangalore ", "PUNE", "pune "])
clean = (raw.str.strip().str.title()
           .replace({"Bangalore": "Bengaluru"}))
print(clean.value_counts())
```

OUTPUT ✓ REAL RUN

```
Bengaluru    2
Pune         2
Name: count, dtype: int64
```

Common mistake Cleaning only the case but not the spaces, so 'Pune' and 'Pune ' still count separately.

Interviewer may ask next How do you split 'Ravi Kumar' into first and last name columns? (str.split(' ', n=1, expand=True).)

11 Create a column based on conditions.

Feature creation · Easy

WHAT THE INTERVIEWER IS CHECKING

np.where / np.select instead of slow row-by-row apply.

MODEL ANSWER

- Two outcomes → np.where(condition, if_true, if_false).
- Several outcomes → np.select(conditions, choices, default).
- pd.cut() is ideal for numeric ranges (bins).

PYTHON

```
sales["size"] = np.where(sales["amount"] >= 1000, "Large", "Small")
conds = [sales["amount"] >= 1500, sales["amount"] >= 800]
sales["tier"] = np.select(conds, ["Gold", "Silver"], default="Bronze")
print(sales[["amount", "size", "tier"]].head(4))
```

OUTPUT ✓ REAL RUN

```
   amount   size    tier
0  1200.0  Large  Silver
1   800.0  Small  Silver
2  1500.0  Large    Gold
3   400.0  Small  Bronze
```

Common mistake NaN amounts silently fall into 'Small'/'Bronze' — handle missing values first.

Interviewer may ask next Do the same banding with pd.cut().

Grouping, merging & reshaping

12 Find total, average and number of orders per city.

GroupBy Easy

WHAT THE INTERVIEWER IS CHECKING

groupby + agg with clear column names — the bread and butter of analyst work.

MODEL ANSWER

- Split by city, apply aggregations, combine — that's groupby.
- Named aggregation gives clean column names in one step.
- Sort the result so the answer is easy to read.

PYTHON

```
summary = (sales.groupby("city")
               .agg(orders=("order_id", "count"),
                    revenue=("amount", "sum"),
                    avg_order=("amount", "mean"))
               .sort_values("revenue", ascending=False)
               .round(1))
print(summary)
```

OUTPUT ✓ REAL RUN

```
           orders  revenue  avg_order
city
Bengaluru       4   5600.0     1400.0
Pune            2   1750.0      875.0
Delhi           2    400.0      400.0
```

Common mistake Using count on a column with NaN — Delhi has 2 orders but only 1 amount. Count the ID column.

Interviewer may ask next What's the difference between agg and transform? (transform returns a value for every original row.)

13 How do merge, join and concat differ?

Merging · Easy

WHAT THE INTERVIEWER IS CHECKING

Mapping SQL joins to pandas.

MODEL ANSWER

- merge: SQL-style join on columns — how='inner' | 'left' | 'right' | 'outer'.
- join: merge on the index (a shortcut).
- concat: stacks tables on top of each other (or side by side) — no key matching.
- indicator=True shows where each row came from — great for checking joins.

PYTHON

```
targets = pd.DataFrame({"city": ["Bengaluru", "Pune", "Mumbai"],
                        "target": [5000, 2000, 1500]})
rev = sales.groupby("city", as_index=False)["amount"].sum()
m = rev.merge(targets, on="city", how="outer", indicator=True)
print(m)
```

OUTPUT ✓ REAL RUN

```
        city  amount  target      _merge
0  Bengaluru  5600.0  5000.0        both
1      Delhi   400.0     NaN   left_only
2     Mumbai     NaN  1500.0  right_only
3       Pune  1750.0  2000.0        both
```

Common mistake Merging on a key that isn't unique on either side — rows multiply. Check row counts after every merge.

Interviewer may ask next How would you find cities with a target but no sales? (Filter _merge == 'right_only'.)

14 Make a city × category revenue table.

Reshaping Medium

WHAT THE INTERVIEWER IS CHECKING

pivot_table for Excel-style summaries.

MODEL ANSWER

- pivot_table(index=rows, columns=columns, values=..., aggfunc=...).
- fill_value=0 replaces empty combinations; margins=True adds totals.

PYTHON

```
pt = sales.pivot_table(index="city", columns="category", values="amount",
                       aggfunc="sum", fill_value=0, margins=True, margins_name="Total")
print(pt)
```

OUTPUT ✓ REAL RUN

```
category    Books  Electronics  Grocery   Total
city
Bengaluru  1500.0       4100.0      0.0  5600.0
Delhi         0.0          0.0    400.0   400.0
Pune       1750.0          0.0      0.0  1750.0
Total      3250.0       4100.0    400.0  7750.0
```

Common mistake Using pivot() (no aggregation) when there are repeated city–category pairs → error.

Interviewer may ask next Show the same table as % of each city's total.

15 Convert a wide table (one column per month) to long format.

Reshaping Medium

WHAT THE INTERVIEWER IS CHECKING

Understanding tidy data — needed for charts and groupby.

MODEL ANSWER

- melt turns columns into rows: id_vars stay, the rest become variable/value pairs.
- pivot / pivot_table does the reverse (long → wide).

PYTHON

```
wide = pd.DataFrame({"city": ["Pune", "Delhi"],
                     "Jan": [800, 0], "Feb": [0, 400], "Mar": [950, 400]})
long = wide.melt(id_vars="city", var_name="month", value_name="revenue")
print(long.head(4))
```

OUTPUT ✓ REAL RUN

```
    city month  revenue
0   Pune   Jan      800
1  Delhi   Jan        0
2   Pune   Feb        0
3  Delhi   Feb      400
```

Common mistake Plotting or grouping wide data by hand, month column by month column.

Interviewer may ask next Why do most plotting libraries prefer long format?

16 Get the top order in each category.

GroupBy Medium

WHAT THE INTERVIEWER IS CHECKING

The pandas version of 'top N per group'.

MODEL ANSWER

- Sort by amount descending, then groupby().head(n) keeps the first n rows of each group.
- Alternative: rank within the group and filter on rank (handles ties your way).

PYTHON

```
top = (sales.sort_values("amount", ascending=False)
            .groupby("category")
            .head(1)[["category", "customer", "amount"]])
print(top)
```

OUTPUT ✓ REAL RUN

```
      category customer  amount
5  Electronics    Pooja  2200.0
2        Books     Ravi  1500.0
3      Grocery    Imran   400.0
```

Common mistake Using groupby().max() — it gives the max of every column separately, mixing values from different rows.

Interviewer may ask next Include ties too (use rank(method='dense') == 1).

Dates, trends & quick analysis

17 Find monthly revenue from an order-date column.

Dates Medium

WHAT THE INTERVIEWER IS CHECKING

to_datetime + the .dt accessor — dates show up in nearly every analyst task.

MODEL ANSWER

- Convert with pd.to_datetime first — string dates can't be grouped by month correctly.
- .dt.to_period('M') (or .dt.month, .dt.day_name()) pulls out parts of the date.
- resample('MS') does the same when the date is the index.

PYTHON

```
sales["order_date"] = pd.to_datetime(sales["order_date"])
monthly = sales.groupby(sales["order_date"].dt.to_period("M"))["amount"].sum()
print(monthly)
print(sales["order_date"].dt.day_name().value_counts().index[0])
```

OUTPUT ✓ REAL RUN

```
order_date
2026-01    2000.0
2026-02    1900.0
2026-03    3850.0
Freq: M, Name: amount, dtype: float64
Monday
```

Common mistake Sorting date strings in DD-MM-YYYY format — they sort alphabetically, not by date.

Interviewer may ask next How many days between each customer's first and last order? (groupby + max - min.)

18 Show the month-over-month % change and a running total.

Trends Medium

WHAT THE INTERVIEWER IS CHECKING

pct_change, cumsum and rolling for trend analysis.

MODEL ANSWER

- pct_change(): change versus the previous row, as a fraction.
- cumsum(): running total. rolling(n).mean(): moving average over n rows.

PYTHON

```
monthly = pd.Series([2000, 1900, 3850], index=["Jan", "Feb", "Mar"])
report = pd.DataFrame({
    "revenue": monthly,
    "mom_%": (monthly.pct_change() * 100).round(1),
    "running": monthly.cumsum(),
    "avg_2m": monthly.rolling(2).mean(),
})
print(report)
```

OUTPUT ✓ REAL RUN

```
     revenue  mom_%  running  avg_2m
Jan     2000    NaN     2000     NaN
Feb     1900   -5.0     3900  1950.0
Mar     3850  102.6     7750  2875.0
```

Common mistake Calculating % change on unsorted dates — always sort by date first.

Interviewer may ask next Why is the first row NaN? (There's no previous month to compare with.)

19 What share of orders does each category have?

Analysis Easy

WHAT THE INTERVIEWER IS CHECKING

value_counts(normalize=True) — quick and very commonly used.

MODEL ANSWER

- value_counts() counts each value, sorted high to low.
- normalize=True turns the counts into proportions; multiply by 100 for %.
- dropna=False includes missing values as their own group.

PYTHON

```
share = sales["category"].value_counts(normalize=True).mul(100).round(1)
print(share)
```

OUTPUT ✓ REAL RUN

```
category
Electronics    37.5
Books          37.5
Grocery        25.0
Name: proportion, dtype: float64
```

Common mistake Reporting counts when the question asked for percentages (or the other way round).

Interviewer may ask next Show revenue share instead of order share. (groupby sum / total.)

20 How do you detect outliers in a numeric column?

Analysis Medium

WHAT THE INTERVIEWER IS CHECKING

The IQR rule and a sensible decision on what to do next.

MODEL ANSWER

- IQR rule: Q1 − 1.5×IQR and Q3 + 1.5×IQR; values outside are flagged.
- Z-score (|z| > 3) works for roughly normal data.
- Then decide: data error → fix/remove; real but rare → keep and report separately.

PYTHON

```
amt = pd.Series([1200, 800, 1500, 400, 700, 2200, 950, 15000])
q1, q3 = amt.quantile([0.25, 0.75])
iqr = q3 - q1
low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
print(f"bounds: {low:.0f} to {high:.0f}")
print(amt[(amt < low) | (amt > high)].tolist())
```

OUTPUT ✓ REAL RUN

```
bounds: -575 to 3025
[15000]
```

Common mistake Deleting outliers automatically — a ₹15,000 order may be your most important customer.

Interviewer may ask next How would you show outliers visually? (Box plot.)

BEFORE YOUR INTERVIEW

Revision checklist

Tick a question only when you can answer it out loud in under two minutes and give one real example.

- [ ] 01 List vs tuple vs set vs dictionary — when do you use each?
- [ ] 02 Use a list comprehension to clean and filter a list.
- [ ] 03 Count how often each word appears in a sentence.
- [ ] 04 What is a lambda function? Where would an analyst use one?
- [ ] 05 You get a new dataset. What are the first things you check?
- [ ] 06 What's the difference between loc and iloc?
- [ ] 07 Filter rows with more than one condition.
- [ ] 08 How do you handle missing values?
- [ ] 09 How do you find and remove duplicate rows?
- [ ] 10 Clean a messy text column (spaces, cases, typos).
- [ ] 11 Create a column based on conditions.
- [ ] 12 Find total, average and number of orders per city.
- [ ] 13 How do merge, join and concat differ?
- [ ] 14 Make a city × category revenue table.
- [ ] 15 Convert a wide table (one column per month) to long format.
- [ ] 16 Get the top order in each category.
- [ ] 17 Find monthly revenue from an order-date column.
- [ ] 18 Show the month-over-month % change and a running total.
- [ ] 19 What share of orders does each category have?
- [ ] 20 How do you detect outliers in a numeric column?

Think out loud

Say your approach before you write anything. Interviewers score your reasoning, not just the final answer.

Connect it to the business

End answers with what the number means: “so we should focus on Bengaluru”, “so the model is safe to launch”.

Know your own projects

Expect “where did you use this?” after almost every question. Keep one project story ready per topic.

Say what you'd check

Mention nulls, duplicates, leakage or edge cases before being asked — it signals real experience.

Prepared the questions?
Now land the interview.

FirstOffer lists fresh fresher jobs, internships and off-campus drives every day — with direct HR emails, Google Forms and official apply links. No middlemen.

firstoffer.online

Daily job alerts on our WhatsApp Channel · Questions vary by company and role; understand the concepts, don't just memorise answers.